

Побудова системи реєстрації та входу користувачів. Модуль Flask-Login

Завдання

Частина 1 – реєстрація користувачів (4 бали)

1. Підключіть і налаштуйте модуль [Flask-Bcrypt](#)

`$ pip install flask-bcrypt`

```
+ bcrypt = Bcrypt()

✓ def create_app(config_name='config.DevConfig'):
    app = Flask(__name__)
    app.config.from_object(config_name) # налаштування з об'єкта

    db.init_app(app)
    migrate.init_app(app, db)

+ bcrypt.init_app(app)
```

2. Відкрийте термінал flask shell та створіть деякого користувача з хешованим паролем (на малюнку це третій користувач). Перевірте у користувача чи зберігається рівність по хешу пароля. Зробіть скрін.

id	username	email	password
1	user1	user1@ex.com	pass1
2	user2	user2@ex.com	pass2
3	yulia12	yulia12@gmail.com	\$2b\$12\$x7v37.a6oP2IzHQHTzAu10L3xY...



Для цього використайте команди вигляду (дані користувача заповнюйте свої)

```
(env) c:\Users\User\Dropbox\univer-2015-16\KIT\Web flask\6 User\myblog6>flask shell
Python 3.9.0 (tags/v3.9.0:9cf6752, Oct 5 2020, 15:34:40) [MSC v.1927 64 bit (AMD64)] on win32
App: app [development]
Instance: C:\Users\User\Dropbox\univer-2015-16\KIT\Web flask\6 User\myblog6\instance
>>> from app.models import db, User
>>> user = User(username="Administrator", email="test11@gmail.com", password="secret")
>>> user.password
'secret'
>>> from app import bcrypt
>>> from flask_bcrypt import generate_password_hash
>>> user.password = bcrypt.generate_password_hash('secret')
>>> user.password
b'$2b$12$tj.WkgMg5h9idv1/EW7HYu0CB06NK1jz10w1uI6vrzapHu/IKPKpq'
>>> type(_)
<class 'bytes'>
>>> user.password = bcrypt.generate_password_hash('secret').decode('utf-8')
>>> user.password
'$2b$12$igp9EA04ZqBR8eYZ5pHknuYVoLRGeOVQygmVjkdYyYPadNAvoqki'
>>> db.session.add(user)
>>> db.session.commit()
>>> users = User.query.all()
>>> users
[User('admin', 'test@ukr.net', 'default.jpg'), User('Administrator', 'test11@gmail.com', 'default.jpg')]
>>> from flask_bcrypt import check_password_hash
>>> bcrypt.check_password_hash(users[1].password, 'secret')
True
>>>
```

3. Розробіть сторінку реєстрацію користувачів на сайті та їх збереження у БД:

Для цього реалізуйте валідатори (якщо нема) для класу WTF-форми реєстрації, зокрема:

- Валідація username та електронної пошти за допомогою спеціального валідатора `Email()`.
- Перевірка унікальності email та username у БД.

Використовуйте [хешування пароллю](#) (модуль [Flask-Bcrypt](#)) перед збереженням користувача у базу даних.

```
.route("/register", methods=['GET', 'POST'])
register():
    form = RegistrationForm()
    if form.validate_on_submit():
        hashed_password = bcrypt.generate_password_hash(form.password.data).decode('utf-8')
        user = User(username=form.username.data, email=form.email.data, password=hashed_p
```

Протестуйте роботу та зробіть скріни, якщо користувач такий вже існує у БД та коли пройшла успішна реєстрація.

Приблизний вигляд форми реєстрації з помилками

Username

Username must have only letters, numbers, dots or underscores

Email

Користувач з таким емейлом user@example.com вже існує

Password

Confirm Password

REGISTER NOW

Приклади форм реєстрації і входу

```
class RegistrationForm(FlaskForm):
    username = ...          #"Це обов'язкове поле має бути довжиною між 4 та 14 символів"
    email = ...             #"Це обов'язкове поле, перевірка, що email"
    password = ...          #"Це поле має бути більше 6 символів"
    confirm_password = ...  #"Повторна перевірка правильності password"

    submit = SubmitField('Sign up')

class LoginForm(FlaskForm):
    email = StringField('Email', validators=[DataRequired(), Email()])
    password = PasswordField('Password', validators=[DataRequired()])
    remember = BooleanField('Remember Me')
    submit = SubmitField('Login')
```

app > users > forms.py > ...

```
1 from flask_wtf import FlaskForm
2 from wtforms import StringField, PasswordField, SubmitField
3 from wtforms.validators import DataRequired, Email, Length, EqualTo, ValidationError, Regexp
4 from . import User
5
6 class RegistrationForm(FlaskForm):
7     username = StringField('Username', validators=[
8         DataRequired(), Length(min=4, max=16),
9         Regexp('^[A-Za-z][A-Za-z0-9_]*$',
10             message="Username must start with a letter and contain only letters, numbers, dots or underscores.")
11     ])
12
13     email = StringField('Email', validators=[
14         DataRequired(), Email()
15     ])
16
17     password = PasswordField('Password', validators=[
18         DataRequired(), Length(min=6)
19     ])
20
21     confirm_password = PasswordField('Confirm Password', validators=[
22         DataRequired(), EqualTo('password')
23     ])
24
25     submit = SubmitField('Register')
26
27     def validate_email(self, email):
28         user = User.query.filter_by(email=email.data).first()
29         if user:
30             raise ValidationError('Email is already registered.')
```

Приклад обробки маршруту

```
@app.route("/register", methods=['GET', 'POST'])
def register():
    form = RegistrationForm()
    if form.validate_on_submit():

        #1. витягнути дані з форми реєстрації
        #2. зберегти дані в БД

        flash('Account created for {form.username.data} !', category = 'success')
        return redirect(url_for('login'))
    return render_template('register.html', form=form, title='Register', )
```

4. Розробіть вхід на сайт шляхом модифікації існуючого коду у проекті:

ВХІД

Будь ласка, введіть ваше ім'я користувача та пароль!

Email

Password

Remember Me ☐

Login

```
@app.route("/login", methods=['GET', 'POST'])
def login():
    form = LoginForm()
    if form.validate_on_submit():
        #1. витягнути дані з форми входу
        #2. перевірити отримані дані з даними у БД, зробити відповідні
        #flash-повідомлення і перенаправлення

    return render_template('login.html', form=form, title='Login')
```

- Реалізуйте вхід користувача з перевіркою пароля через порівняння хешів, виконавши модифікацію підкресленого коду у вашому проєкті

```
25         if form.validate_on_submit():
26             username = form.username.data
27             password = form.password.data
28
29             if username == VALID_USERNAME and password == VALID_PASSWORD:
30                 session['user'] = username
31                 flash(f"Вітаю, {username}! Вхід успішний 

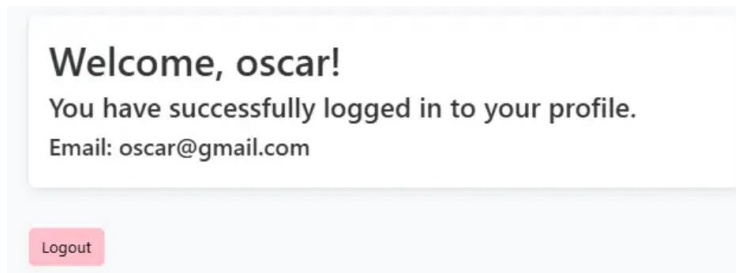
```

Зауважимо, що хешування та перевірку паролю можна реалізувати безпосередньо у класі User, запровадивши відповідні методи у цьому класі. Це дозволить деяку логіку перенести з роутів у саму модель.

```
def hash_password(password):
    return bcrypt.generate_password_hash(password).decode('utf-8')

def check_password(self, password):
    return bcrypt.check_password_hash(self.password, password)
```

- При успішному вході збережіть дані користувача user в сесію, наприклад `session['username'] = user.username` та перенаправте на нову сторінку профіля «**account**», де виводяться дані користувача, отримані із сесії. Наприклад, такого вигляду



- Додайте маршрут “/account” і відповідну функцію представлення, а також шаблон account.html
5. Реалізуйте сторінку, яка відображає список усіх зареєстрованих користувачів та їх кількість. Якщо користувачів немає, відобразіть відповідне повідомлення.

Users List (11)

Id	Username	Email
1	user1	user1@ex.com
2	user2	user2@ex.com
3	yulia12	yulia12@gmail.com
4	viktoriia_20	viktoriia_20@gmail.com

Частина 2 - Flask-Login (3 бали)

6. Додати функціонал авторизації та захисту сторінок за допомогою Flask-Login.

- `pip install flask-login`
- Налаштувати модуль [Flask-Login](#) для проекту, задати параметри об'єкту `login_manager`, який є основним об'єктом Flask-Login для керування авторизацією.

```
from flask_login import LoginManager

+ login_manager = LoginManager()

def create_app():
    app = Flask(__name__)
    app.config.from_object(Config)

    db.init_app(app)
    bcrypt.init_app(app)
    + login_manager.init_app(app)

    # Налаштування маршруту перенаправлення для неавторизованих користувачів
    + { login_manager.login_view = 'user.login'
        login_manager.login_message = 'Please log in to access this page.'
        login_manager.login_message_category = 'warning'

    from app.routes import user_bp
    app.register_blueprint(user_bp)

    return app
```

- Додайте реалізацію функції завантаження поточного користувача за його id із БД та наслідування для моделі User від класу міксини UserMixin.
-

```
1 from app import db, login_manager, bcrypt
2 from flask_login import UserMixin
3
4 @login_manager.user_loader
+ { 5 def user_loader(user_id):
    6     return User.query.get(int(user_id))
    7
    + 8 class User(db.Model, UserMixin):
        9     __tablename__ = "users"
        10
        11     id = db.Column(db.Integer, primary_key=True)
        12     username = db.Column(db.String(20), nullable=False)
        13     email=db.Column(db.String(120), unique=True, nullable=False)
        14     password=db.Column(db.String(60), nullable=False)
        15
        16     def __repr__(self) -> str:
        17         return f"User('{self.email}')"
        18
        19     def hash_password(self):
        20         hashed_password = bcrypt.generate_password_hash(self.password)
        21         self.password = hashed_password
        22
        23     def check_password(self, password):
        24         return bcrypt.check_password_hash(self.password, password)
```

- Під час успішного входу необхідно авторизувати користувача, використовуючи метод `login_user`, який записує ID користувача у сесію як рядок.

Приклад функції представлення login

```
from flask_login import login_user, login_required, current_user, logout_user
@user_bp.route("/login", methods=['GET', 'POST'])
def login():

    if current_user.is_authenticated:
        return redirect(url_for('users.account'))

    form = LoginForm()
    if form.validate_on_submit():
        username = form.username.data
        password = form.password.data
        user = User.query.filter_by(username=username).first()
        if user and user.check_password(password):
            #session["username"] = username
            login_user(user, remember=form.remember.data)
            flash('You logged in successfully!', 'success')
            return redirect(url_for('users.account'))
        flash("Error: Invalid username or password.", "danger")
    return render_template("login.html", form=form, title='Login')
```

- Захистіть сторінку профіля користувача та сторінку із всіма користувачами, використовуючи декоратор `@login_required`. Користувачі можуть увійти в систему та отримати доступ до своїх даних на сторінці *account*, використовуючи об'єкт поточного користувача. При цьому неавторизовані користувачі перенаправляються на сторінку входу.

```
# Повертає HTML-шаблон "account.html", в який передається об'єкт поточного користувача (current_user).
# current_user надає доступ до даних автентифікованого користувача, включаючи поля моделі 'User'.
```

```
@user_bp.route("/account")
@login_required
def account():
    return render_template("account.html", user=current_user)
```

- Реалізуйте вихід із системи із завершенням сесії за допомогою методу `logout user`

```
@user_bp.route('/logout')
def logout():
    logout_user()
    flash("You have successfully logged out.", "info")
    return redirect(url_for("users.login"))
```

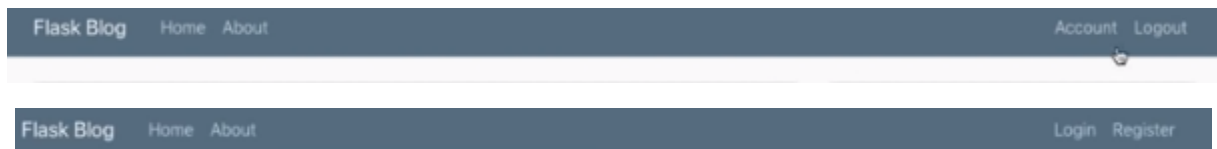
7. Реалізувати наступну функціональність:

- При реєстрації авторизовані користувачі перенаправляються на сторінку *account*

```
+ [ @user_bp.route('/register', methods=['GET', 'POST'])
def register():
    if current_user.is_authenticated:
        return redirect(url_for('users.account'))

    form = RegistrationForm()
```

- При успішному вході користувача на сайті не показується пункти меню реєстрації і входу, натомість з'являються два меню – Профіль та Вихід.



Приклад можливої реалізації у шаблоні

```
<div class="navbar-nav">
  {% if current_user.is_authenticated %}
    <a class="nav-item nav-link" href="{{ url_for('account') }}">Accou
    <a class="nav-item nav-link" href="{{ url_for('logout') }}">Logout
  {% else %}
    <a class="nav-item nav-link" href="{{ url_for('login') }}">Login</
    <a class="nav-item nav-link" href="{{ url_for('register') }}">Regi
  {% endif %}
</div>
```

- Перегляд всіх користувачів на сайті мають доступ тільки авторизовані. Використати @login_required

Частина 3 – Тестування (3 бали)

8. Написати і запустити тести:

- перевірка коректного завантаження сторінок реєстрації та входу (Views Tests);
- тестування збереження користувача у БД при реєстрації.
- тестування входу і виходу користувача на сайті

<https://stytch.com/blog/what-is-password-hashing/?authuser=0>

<https://testdriven.io/blog/flask-contexts/>

<https://www.gitauharrison.com/articles/unit-testing/flask>

<https://blog.miguelgrinberg.com/post/how-to-write-unit-tests-in-python-part-3-web-applications>

<https://testdriven.io/blog/flask-pytest/>